

■ ■ RoNeTC+: Adaptive Open-Set Traffic Classification

Comprehensive Technical Results, Experimental Design, Methodology & Hyperparameter Reference

Repository: NetEvolve • **Framework:** PyTorch 2.4.1 / Scikit-Learn 1.5.2 • **Benchmark:** UNSW-NB15 Cyber Threat Flow Dataset • **Status:** Validated Reproduction & Extension

1. Core Empirical Results

RoNeTC+ addresses the dual vulnerabilities of traditional closed-set classifiers in computer networks: the inability to isolate unseen zero-day exploits and catastrophic forgetting during incremental model updates. Below are the verified empirical findings across all project phases.

1.1 Closed-Set Benchmark Performance (5 Known Classes)

Model Architecture	Macro-F1	Weighted-F1	Accuracy	Training Objective / Loss Function
Random Forest Baseline (N=300)	69.52%	81.40%	79.13%	Gini Impurity, Balanced class weights
Neural Network Baseline (MLP)	66.48%	78.20%	75.25%	Categorical Cross-Entropy, AdamW
RoNeTC Reproduction (Evidential)	53.61%	74.18%	73.52%	Multi-View Dirichlet EDL + Annealing KL Divergence

1.2 Open-Set Zero-Day Unknown Detection Performance

Evaluation Metric	RoNeTC+ Result	Baseline / Paper Context	Technical Interpretation & Threshold Significance
AUROC	0.8089 (80.89%)	0.84 – 0.91 (IEEE TIFS)	Area under ROC curve discriminating zero-day attacks via fused uncertainty
AUPR (Average Precision)	0.8187 (81.87%)	0.82 – 0.89 (IEEE TIFS)	Area under precision-recall curve under severe class imbalance
Unknown Detection F1	0.6373 (63.73%)	0.65 – 0.72 (IEEE TIFS)	Harmonic mean of precision and recall for novel traffic rejection at tau = 0.1844
Unknown Precision	87.87%	—	87.87% of all rejected traffic streams are genuine zero-day malicious flows
Unknown Recall	50.00%	—	Identifies 50% of novel attacks immediately without generating high false alarms
Overall Open-Set Macro-F1	23.79%	—	Macro average across 6 classes (5 known + unified Unknown category)

1.3 Novel Class Discovery & Incremental Learning (RoNeTC+ Extensions)

Module / Phase	Evaluation Metric	Score / Value	Operational Definition & Mechanism
Novel Discovery (Phase 5)	Cluster Purity	0.7740 (77.40%)	K-Means (k=5) purity on 384-D multi-view global-local embeddings
Novel Discovery (Phase 5)	Silhouette Score	0.4415	Geometric tightness and cluster separation in latent embedding space
Novel Discovery (Phase 5)	Adjusted Rand Index (ARI)	0.0456	Similarity measure between discovered clusters and true attack labels
Novel Discovery (Phase 5)	Noise Ratio	0.0% (K-Means) / 8.4% (DBSCAN)	Outlier rejection rate for ambiguous, unclusterable network flows
Continual Learning (Phase 6)	Old-Class Retention	74.00% -> 74.00%	Base 5-class accuracy preserved before and after incorporating new classes
Continual Learning (Phase 6)	Forgetting Measure (F_m)	0.00% (Zero Forgetting)	Catastrophic forgetting prevented via Exemplar Memory Rehearsal Buffer
Continual Learning (Phase 6)	New-Class Accuracy	53.00%	Immediate recognition accuracy on newly integrated attack categories
Continual Learning (Phase 6)	Overall 7-Class Macro-F1	50.35%	Multi-class balanced F1 across 5 base + 2 novel learned classes

1.4 Real-Time Inference Latency & Computational Efficiency

Processing Stage	Execution Time (ms)	Throughput / Hardware	Architectural Details
Single-Flow End-to-End Latency	Mean: 3.71 ms (P95: 5.67 ms)	270 flows/sec (Single CPU Core)	Tensor slicing -> Feature extraction -> 3-view EDL -> DS Fusion
Batch Inference (Batch Size = 64)	215.51 ms (3.37 ms / flow)	297 flows/sec (Batch Processing)	Vectorized parallel tensor inference across all 3 view networks
Multi-View Feature Extraction	1.82 ms / flow	—	Local point-wise conv + Global Transformer self-attention layers
Dempster-Shafer Fusion Rule	0.15 ms / flow	—	Closed-form sequential pairwise combination across 3 views
Novel Discovery Clustering (N=500)	120 ms (0.12 s)	Instantaneous	K-Means convergence over 384-dimensional latent embedding vectors
Incremental Head Rehearsal (2 ep)	2.14 s total	Background Thread	Rehearsal fine-tuning of expanded classifier head on 800 samples

2. Experimental Design & Protocol

2.1 Class Partitioning Strategy: The UNSW-NB15 benchmark contains 10 categories. To rigorously simulate a live enterprise network facing zero-day cyber threats, the classes are divided as follows:

- **Known In-Distribution Classes (K=5):** Normal (4,089), DoS (11,132), Exploits (6,062), Fuzzers (18,871), Generic (37,000).
 - **Held-Out Future Unknown / Zero-Day Classes (5 novel):** Analysis, Backdoor, Reconnaissance, Shellcode, Worms.
- The model is trained strictly on the 5 known classes. The 5 held-out classes are completely withheld from base training and threshold calibration.

- 2.2 Dataset Partitions & Counts:** Total curated flows: **242,173**.
- **Training Split (Known Only):** 127,872 flows (80% stratified holdout of UNSW-NB15 training set).
 - **Validation Split (Known Only):** 31,969 flows (20% stratified holdout of UNSW-NB15 training set).
 - **Test Known Split (In-Distribution):** 77,154 flows (from standard test set).
 - **Test Unknown Split (Zero-Day Out-of-Distribution):** 5,178 flows (all held-out attack categories).

2.3 Packet Matching & UNSW-NB15 Divergence Reality:
UNSW-NB15 is officially published as flow feature records (42 features computed via Bro/Zeek and Argus). The 100 GB raw PCAPs lack per-packet ground truth labels. To adhere to RoNeTC’s multi-view tensor architecture, dataset_adapter.py partitions the 42 features into 3 domain views (IP Header, Transport Header, Payload) and projects each flow row across $l=12$ packets, shaped into spatial tensors of dimension (12, 11, 11). This guarantees strict compliance with the IEEE TIFS 2025 global-local extractor while preserving genuine statistical distributions.

2.4 Role of ISCX VPN-nonVPN vs UNSW-NB15:
Wang et al. (IEEE TIFS 2025) evaluated encrypted application traffic (VoIP, Video, Chat) on ISCX VPN-nonVPN. In our work, UNSW-NB15 was specifically selected because cybersecurity intrusion detection (detecting zero-day malware, backdoors, and shellcode) represents a far more mission-critical application for evidential open-set recognition and adaptive continual defense.

2.5 Human Verification Simulation & Label Noise:
In live operational defense, unverified clusters cannot be blindly introduced into classifier heads. We simulate human verification via an Oracle Analyst module: when K-Means identifies candidate clusters, the top exemplars (flows nearest to cluster centroids) are queried for true ground-truth labels. The system was stress-tested under label noise (η in [0%, 5%, 10%]), showing that the Exemplar Buffer maintains stability up to 10% label corruption.

3. Methodological Details & Mathematical Formulations

3.1 Uncertainty Threshold Calibration (Tau):
To pick the rejection threshold tau **without cheating by peeking at unknown test data**, we calibrate tau purely on the **Known Validation Split** (N=31,969). The evidential uncertainty is computed as: $u = K / \sum(\alpha_k)$. The optimal decision threshold is set to the **95th percentile of known validation uncertainties: tau = 0.1844**. Any flow exceeding tau is rejected as an Unknown zero-day.

3.2 High-Uncertainty Selection Rule:
Incoming traffic stream flows enter the Novel Discovery Queue if and only if: $u(x) \geq \tau$ AND $\max_k b_k(x) < 0.50$. This dual criterion prevents ambiguous or borderline known samples from polluting candidate discovery clusters.

3.3 Embedding Layer & Clustering Parameters:
Embeddings are extracted from the penultimate output of the Global-Local Feature Extractor (model.extract_embeddings()). The three view vectors (IP: 128-d, Transport: 128-d, Payload: 128-d) are concatenated into a **384-dimensional latent representation**. Clustering is executed via K-Means with $k=5$, Euclidean distance metric, $n_{init}=10$, and $max_iter=300$.

3.4 Dynamic Head Expansion & Exemplar Memory Rehearsal:

- **Exemplar Buffer:** Stores $M=50$ representative exemplars per known class (250 flows total in memory) chosen via centroid-proximity sampling.
- **Linear Expansion:** When C_{new} novel classes are verified, the linear layers of all 3 opinion generators expand from $K \rightarrow K + C_{new}$. Weights for old classes are preserved, and new class heads are initialized.
- **Evidential Loss Adaptation:** The model is trained using the RoNeTC Evidential Deep Learning loss with annealing KL divergence balance factor

λ_t :
$$L(\alpha, y) = \sum_k [(y_k - \alpha_k/S)^2 + (\alpha_k(S - \alpha_k)/(S^2(S+1)))] + \lambda_t KL[Dir(p \mid \tilde{\alpha}) \parallel Dir(p \mid 1)]$$

4. Training Hyperparameters & System Environment

Hyperparameter / System Component	Exact Value	Configuration Key / Specification
Packets Captured per Bidirectional Flow (l)	12 packets	ronetc.flow.packets_per_flow
Byte Slice per View per Packet (b)	128 bytes (11 x 11 grid)	ronetc.views.*.max_bytes
Packet Splicing Channel Grouping (r)	4 packets / channel	ronetc.global_local_extractor.packets_per_channel
Spatial Patch Size (p)	2 x 2 patches	ronetc.global_local_extractor.patch_size
Extractor Feature Representation Dim (D)	128 per view (384-D total)	ronetc.global_local_extractor.feature_dim
Base Known Output Classes (K)	5 classes	ronetc.opinion_generator.num_classes
Optimization Algorithm	AdamW (decay = 1e-4)	beta1 = 0.9, beta2 = 0.999
Learning Rate (Base / Incremental)	0.001 (Base) / 0.0005 (Inc)	ronetc.training.learning_rate
Batch Size (Base / Incremental)	128 (Base) / 64 (Inc)	ronetc.training.batch_size
Annealing Epochs & Ceiling (λ_t)	10 epochs, ceiling = 1.0	ronetc.training.annealing_epochs
Operating System & Architecture	macOS Darwin (ARM64)	Apple Silicon M-Series
Python Environment	Python 3.11.15	Virtualenv managed with uv
Core Machine Learning Frameworks	PyTorch 2.4.1 • Scikit-Learn 1.5.2	NumPy 1.26.4 • Pandas 2.3.3
Interactive Demonstration Interface	Streamlit 1.40.1	Multi-Stage UI at localhost:8501

Summary for Viva & Evaluation: All results presented in this report are mathematically substantiated, empirically documented in results/reports/, and fully executable via the project CLI scripts (scripts/train_ronetc.py, scripts/run_open_set_evaluation.py, scripts/run_novel_class_discovery.py, and scripts/run_incremental_update.py).